ИТМО

Факультет Систем Управления и Робототехники

ПРАКТИЧЕСКАЯ ЛИНЕЙНАЯ АЛГЕБРА — ЛАБОРАТОРНАЯ РАБОТА №3

Аннотация – В работе исследованы матрицы 4×4 в однородных координатах, применяемые в 3D-графике. Построены матрицы масштабирования, перемещения и поворота (по формуле $R_v(\theta) = e^{J\theta}$), поворота вокруг оси через заданную точку, матрицы положения камеры и перспективы. Все преобразования выполнены матричным умножением над вершинами кубика и сцены из нескольких фигур, результаты визуализированы на Python (`matplotlib`), выполнено необязательное задание 8 (домик).

Ключевые слова: однородные координаты; матрицы преобразований; поворот; матричная экспонента; камера; перспектива

Выполнили

Ануфриев Андрей Сергеевич (465029)

Бахуров Николай Алексеевич (504862)

Раджапов Фёдор Сергеевич (502544)

Проверил

Зарина Ольга Андреевна

2026-10-29

Содержание

1 Постановка задачи **2**

2 Задание 1. Куб **2**

3 Задание 2. Масштабирование **3**

4 Задание 3. Перемещение **4**

5 Задание 4. Вращение вокруг оси **5**

6 Задание 5. Вращение вокруг вершины куба **7**

7 Задание 6. Камера **8**

8 Задание 7. Перспектива **11**

9 Задание 8 (необязательное). Домик **13**

10 Выводы **14**

11 Контрольные вопросы **14**

11.1 Однородные координаты 14

11.2 Виды преобразований, аффинные преобразования 14

11.3 Масштабирование вдоль вектора 15

11.4 Коммутирование S и T 15

11.5 Кососимметрические и симметрические матрицы 15

11.6 Комплексные собственные числа вещественной матрицы 15

11.7 Аналитические функции и матричная экспонента 15

11.8 Собственное число 1 у матриц $SO(3)$ 16

11.9 Ось вращения по матрице поворота 16

11.10 Подобные матрицы 16

11.11 Смысл матрицы C^{-1} 16

11.12 Виды проекций и действие перспективы 16

12 Приложение. Код выполнения **16**

13 Заключение **29**

1 Постановка задачи

Требуется познакомиться с матрицами преобразований пространства в однородных координатах и построить с их помощью: куб и другую фигуру (задание 1), масштабирование (2), перемещение (3), поворот вокруг оси v через начало координат (4) и вокруг вершины куба (5), камеру (6) и перспективу (7), а также домик из блоков (8, необязательное). Все преобразования координат выполняются *только* матричным умножением.

Соглашения. Точка $p = (x, y, z)$ записывается столбцом $(x, y, z, 1)^T$, преобразование — левое умножение $p' = Mp$. Композиция «сначала A, затем B» равна BA. (В одном из источников методички матрицы транспонированы, т. е. используются векторы-строки: $p' = pM^T$.) Камера по умолчанию находится в начале координат и смотрит вдоль $+z$, ось x направлена вправо, ось y — вниз; так выглядит график при `ax.view_init(azim=0, elev=-90)` (при необходимости с `roll=-90`, см. задание 6).

Все числовые матрицы в тексте подставляются из файла `matrices.tex`, который создаёт программа, поэтому числа в отчёте совпадают с расчётом в коде.

2 Задание 1. Куб

Куб задан матрицей вершин 4×8 (столбец — вершина) и списком граней (номера вершин, нумерация с нуля), как в листинге методички:

$$V_{cube} = \begin{pmatrix} -1 & 1 & 1 & -1 & -1 & 1 & 1 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \\ -1 & -1 & -1 & -1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

Функция `draw_shape` берёт первые три строки и делит их на четвёртую: $(x, y, z, w) \mapsto (x/w, y/w, z/w)$, затем строит многоугольники по спискам индексов граней.

Почему четыре компоненты, а не три. Перемещение $p \mapsto p + t$ не является линейным отображением $\mathbb{R}^3$ (нулевой вектор не остаётся нулевым), поэтому его нельзя записать матрицей 3×3. В однородных координатах $(x, y, z, 1)$ оно становится линейным: $\begin{pmatrix} I & t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} p \\ 1 \end{pmatrix} = \begin{pmatrix} p + t \\ 1 \end{pmatrix}$. Благодаря этому масштабирование, поворот, перемещение и перспектива — матрицы одного размера, и любую цепочку преобразований можно заранее перемножить в одну матрицу. Четвёртая компонента $w \neq 1$ нужна для перспективы (деление на w). Векторы с $w = 0$ описывают направления (точки на бесконечности), к ним перемещение не применяется.

Как задать другую фигуру. Нужно заменить матрицу вершин $4 \times N$ и список граней: каждая грань — список номеров вершин в порядке обхода (длины граней могут быть разными). Для пирамиды с квадратным основанием:

$$V_{pyr} = \begin{pmatrix} -1 & 1 & 1 & -1 & 0 \\ -1 & -1 & 1 & 1 & 0 \\ -1 & -1 & -1 & -1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix}, \quad \text{грани: } [0,1,2,3],\ [0,1,4],\ [1,2,4],\ [2,3,4],\ [3,0,4].$$

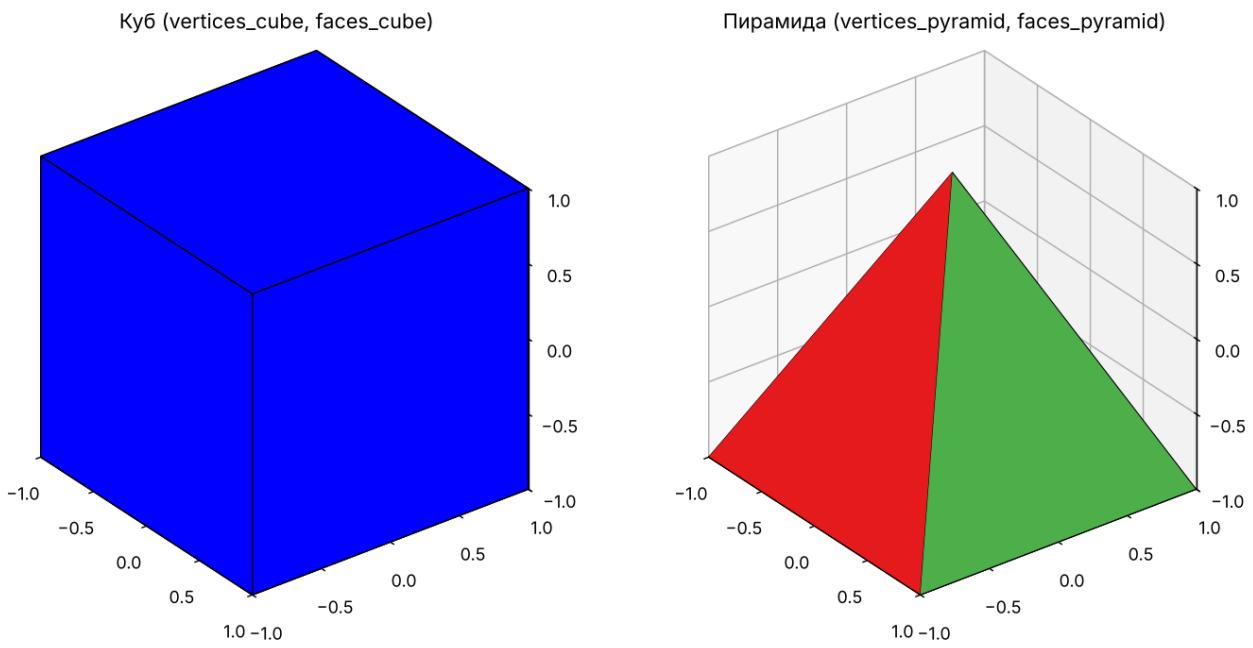


Рис. 1: Задание 1. Результат программы для куба (слева) и для пирамиды (справа)

3 Задание 2. Масштабирование

Общая структура. Масштабирование по осям с коэффициентами s_x, s_y, s_z (центр — начало координат):

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad \det S = s_x s_y s_z.$$

Объём изменяется в $|s_x s_y s_z|$ раз; $s_i < 0$ даёт ещё и отражение; $S^{-1} = S(1/s_x, 1/s_y, 1/s_z)$. Композиция двух масштабирований — снова масштабирование с произведением коэффициентов: $S(a)S(b) = S(a_x b_x, a_y b_y, a_z b_z)$.

Выбранные матрицы.

$$S_1 = S(2, 1, 1.5) = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1.5 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad S_2 = S(0.5, 2, 2) = \begin{pmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix},$$

$$S_2 S_1 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = S(1, 2, 3).$$

Куб растягивается вдоль x и z (для S_1), вдоль y и z с одновременным сжатием вдоль x (для S_2); итоговая матрица оставляет размер по x равным 2, удваивает по y и утраивает по z. Центр куба остаётся в начале координат (на рисунке серым показан исходный куб).

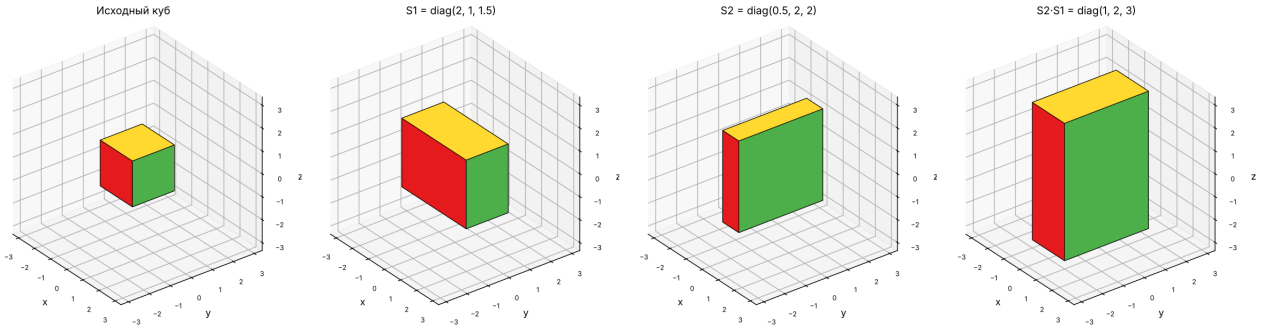


Рис. 2: Задание 2. Исходный куб, образы под действием S_1, S_2 и S_2S_1

4 Задание 3. Перемещение

Общая структура. Перемещение на вектор $t = (t_x, t_y, t_z)^T$:

$$T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} I & t \\ 0 & 1 \end{pmatrix}.$$

Действие: $T(x, y, z, 1)^T = (x + t_x, y + t_y, z + t_z, 1)^T$. Свойства: $T(a)T(b) = T(a + b)$, $T(t)^{-1} = T(-t)$, $\det T = 1$ (объём сохраняется), T не является линейным отображением $\mathbb{R}^3$, а лишь аффинным.

Выбранные матрицы.

$$T_1 = T(2, 1, 0) = \begin{pmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad T_2 = T(-1, 0, 2) = \begin{pmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix},$$

$$T_2T_1 = T(1, 1, 2) = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

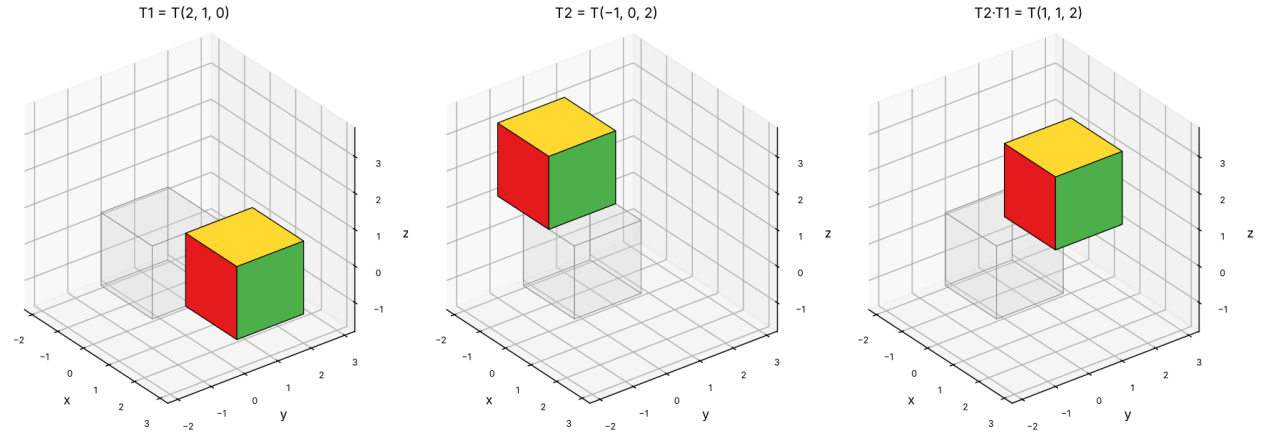


Рис. 3: Задание 3. Образы куба под действием T_1, T_2 и T_2T_1

Комбинации TS и ST. Пусть $S = S_2S_1 = \text{diag}(1,2,3,1)$ и $T = T_2T_1 = T(1,1,2)$. В блочном виде

$$TS = \begin{pmatrix} I & t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} S & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} S & t \\ 0 & 1 \end{pmatrix}, \qquad ST = \begin{pmatrix} S & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} I & t \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} S & St \\ 0 & 1 \end{pmatrix}.$$

Численно:

$$TS = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 2 & 0 & 1 \\ 0 & 0 & 3 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad ST = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 2 & 0 & 2 \\ 0 & 0 & 3 & 6 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Преобразования **не эквивалентны**: $TS = ST \iff St = t \iff (S - I)t = 0$. Здесь $St = (1,2,6)^T \neq t = (1,1,2)^T$. Геометрически: в TS куб сначала растягивается вокруг своего центра, затем сдвигается на t (центр оказывается в $(1,1,2)$); в ST куб сначала сдвигается, а затем вместе со сдвигом растягивается относительно начала координат — сдвиг тоже масштабируется (центр оказывается в $St = (1,2,6)$).

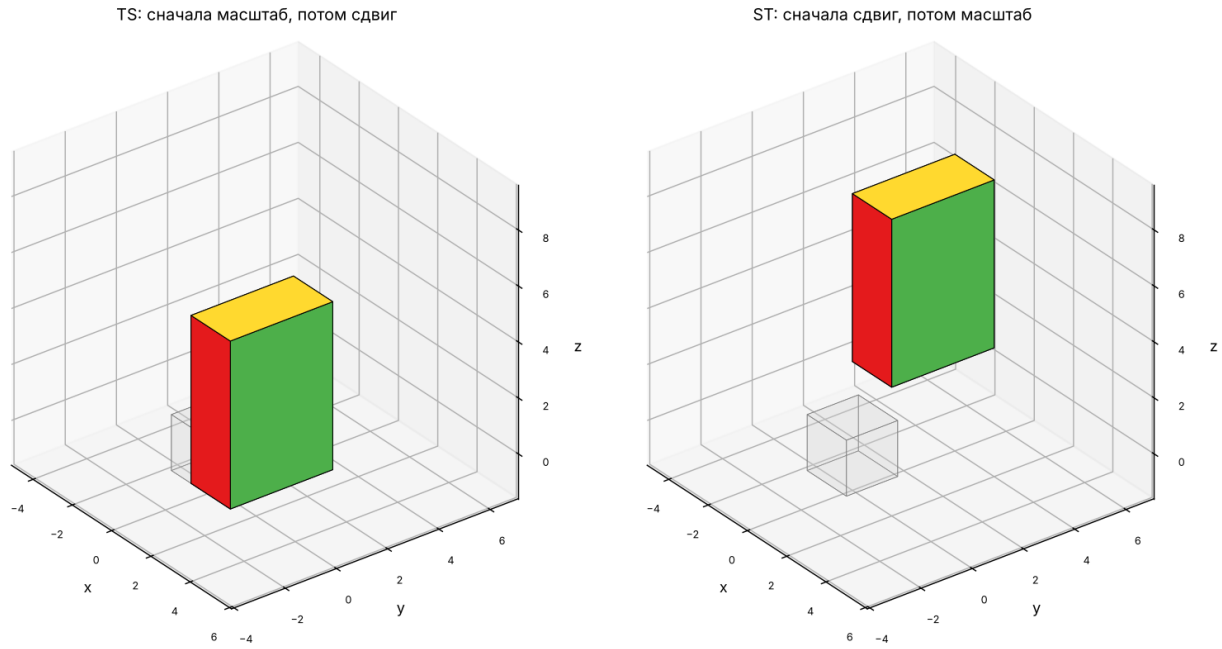


Рис. 4: Задание 3. Слева TS (масштаб, затем сдвиг), справа ST (сдвиг, затем масштаб)

5 Задание 4. Вращение вокруг оси

Общая структура. Пусть $v = (v_x, v_y, v_z)^T$ — ось вращения (через начало координат), $\hat{v} = v/\|v\|$. Матрица

$$J = \begin{pmatrix} 0 & -\hat{v}_z & \hat{v}_y & 0 \\ \hat{v}_z & 0 & -\hat{v}_x & 0 \\ -\hat{v}_y & \hat{v}_x & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

— кососимметрическая матрица векторного произведения: для любого w верно $Jw = \hat{v} \times w$. Поворот на угол θ против часовой стрелки (если смотреть с конца вектора $\hat{v}$): $R_v(\theta) = e^{J\theta}$.

Как работает формула. Функция $\varphi(\theta) = e^{J\theta}p$ — решение уравнения $\dot{x} = Jx = \hat{v} \times x$, то есть движения точки с угловой скоростью $\hat{v}$ (вращение вокруг $\hat{v}$ с единичной угловой скоростью), поэтому $e^{J\theta}$ поворачивает на угол θ. Для $\|\hat{v}\| = 1$ выполнено $J^2 = \hat{v}\hat{v}^T - I$ и $J^3 = -J$ (на

блоке 3×3), поэтому ряд $e^{J\theta}=\sum J^k\theta^k/k!$ сворачивается в формулу Родрига

$$R_v(\theta)=I+\sin\theta\,J+(1-\cos\theta)J^2,\qquad \text{на блоке } 3\times 3:\ \cos\theta\,I+\sin\theta\,J+(1-\cos\theta)\hat v\hat v^T.$$

Ось неподвижна: $J\hat v=\hat v\times\hat v=0$, поэтому $e^{J\theta}\hat v=\hat v$. Матрица R ортогональна, так как $J^T=-J$ и $(e^{J\theta})^T=e^{-J\theta}=(e^{J\theta})^{-1}$, а $\det e^{J\theta}=e^{\mathrm{tr}\,J\theta}=1$. В программе поворот считается и через `scipy.linalg.expm`, и по формуле Родрига, результаты совпадают (проверка `assert`).

Повороты вокруг осей координат ($\hat v=e_x,e_y,e_z$):

$$R_x(\theta)=\begin{pmatrix}1&0&0&0\\0&\cos\theta&-\sin\theta&0\\0&\sin\theta&\cos\theta&0\\0&0&0&1\end{pmatrix},\quad R_y(\theta)=\begin{pmatrix}\cos\theta&0&\sin\theta&0\\0&1&0&0\\-\sin\theta&0&\cos\theta&0\\0&0&0&1\end{pmatrix},$$

$$R_z(\theta)=\begin{pmatrix}\cos\theta&-\sin\theta&0&0\\\sin\theta&\cos\theta&0&0\\0&0&1&0\\0&0&0&1\end{pmatrix}.$$

Выбранные оси и углы. $v_1=(1,2,2)^T$ ($\|v_1\|=3$), $\theta_1=60^\circ$; $v_2=(2,-1,2)^T$ ($\|v_2\|=3$), $\theta_2=45^\circ$. Для первой оси $J_1=\begin{pmatrix}0&-0.6667&0.6667\\0.6667&0&-0.3333\\-0.6667&0.3333&0\end{pmatrix}$ (блок 3×3), и

$$R_1=R_{v_1}(60^\circ)=\begin{pmatrix}0.5556&-0.4662&0.6885&0\\0.6885&0.7222&-0.0665&0\\-0.4662&0.5109&0.7222&0\\0&0&0&1\end{pmatrix},$$

$$R_2=R_{v_2}(45^\circ)=\begin{pmatrix}0.8373&-0.5365&-0.1055&0\\0.4063&0.7397&-0.5365&0\\0.3659&0.4063&0.8373&0\\0&0&0&1\end{pmatrix}.$$

Композиции (результат зависит от порядка):

$$R_2R_1\ (\text{сначала } R_1)=\begin{pmatrix}0.145&-0.8318&0.5359&0\\0.9851&0.0707&-0.1569&0\\0.0926&0.5506&0.8296&0\\0&0&0&1\end{pmatrix},$$

$$R_1R_2\ (\text{сначала } R_2)=\begin{pmatrix}0.5276&-0.3632&0.7679&0\\0.8456&0.1378&-0.5158&0\\0.0815&0.9215&0.3798&0\\0&0&0&1\end{pmatrix}.$$

$R_2R_1\neq R_1R_2$: повороты в 3D не коммутируют. Композиция — снова поворот (она ортогональна, $\det=1$); её ось находится как неподвижный вектор: для R_2R_1 ось $(0.354,\ 0.222,\ 0.909)^T$, для R_1R_2 — $(0.719,\ 0.343,\ 0.605)^T$, а угол обоих поворотов одинаков ($\approx 88.7^\circ$), так как $\mathrm{tr}(R_2R_1)=\mathrm{tr}(R_1R_2)$. На рисунке красным показан вектор вдоль оси вращения (и пунктирная прямая оси).

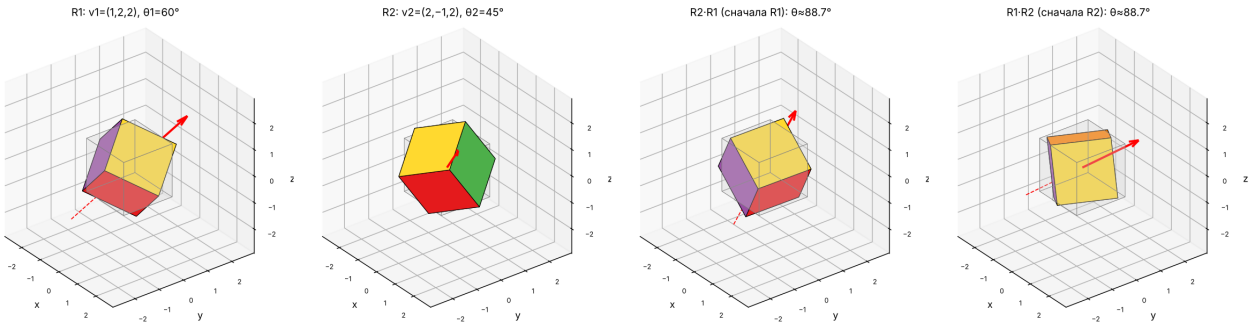


Рис. 5: Задание 4. R_1, R_2 и композиции R_2R_1, R_1R_2 (исходный куб показан серым)

Зависимость от угла и оси показана на рисунках 6 и 7: при $\theta = 180°$ куб переворачивается ($R^2 = I$), при малых θ почти не сдвигается; вращение вокруг осей x, y, z оставляет неподвижным соответствующее направление.

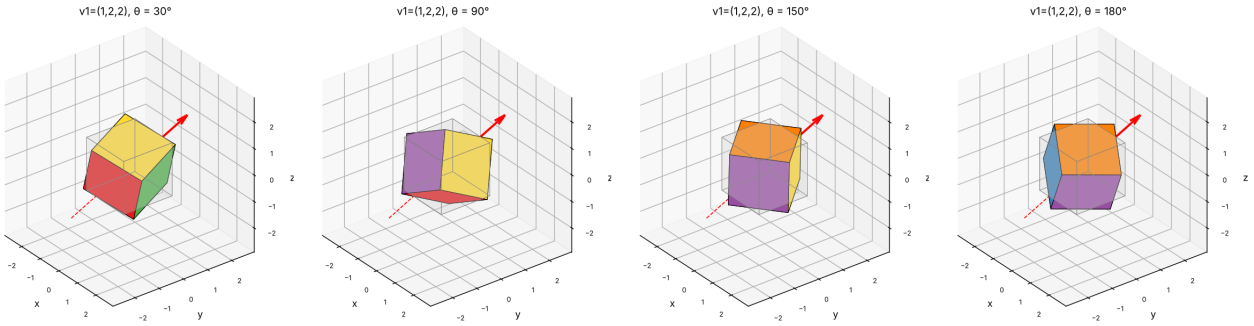


Рис. 6: Задание 4. Поворот вокруг оси v_1 на $30°$, $90°$, $150°$, $180°$

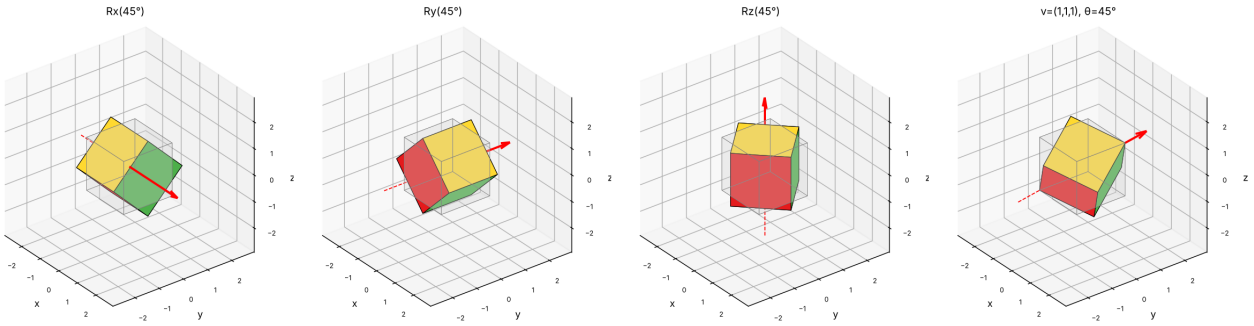


Рис. 7: Задание 4. Поворот на $45°$ вокруг осей x, y, z и диагонали $(1, 1, 1)$

6 Задание 5. Вращение вокруг вершины куба

Общая формула. Поворот вокруг оси с направлением v, проходящей через точку $M = (x_0, y_0, z_0)$: перенести M в начало координат, повернуть вокруг v, вернуть назад:

$$R_{v,M}(\theta) = T(M)\, R_v(\theta)\, T(-M) = \begin{pmatrix} R & M - RM \\ 0 & 1 \end{pmatrix},$$

где R — блок 3×3 матрицы $R_v(\theta)$. Точка M неподвижна: $R_{v,M}M = M$.

Выбранное преобразование. Центр вращения — вершина $M = (1, 1, 1)$, направление оси $v = (1, 1, 0)^T$, угол $\theta = 60°$:

$$R_{v,M}(60°) = \begin{pmatrix} 0.75 & 0.25 & 0.6124 & -0.6124 \\ 0.25 & 0.75 & -0.6124 & 0.6124 \\ -0.6124 & 0.6124 & 0.5 & 0.5 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Блок 3×3 равен $\cos\theta I + \sin\theta J + (1 - \cos\theta)\hat{v}\hat{v}^T$ при $\hat{v} = (1, 1, 0)^T/\sqrt{2}$, последний столбец — $M - RM = (-0.6124,\ 0.6124,\ 0.5)^T$. На рисунке красным показана ось вращения, она проходит через вершину M (чёрная точка); исходный куб показан серым, повёрнутый — цветом.

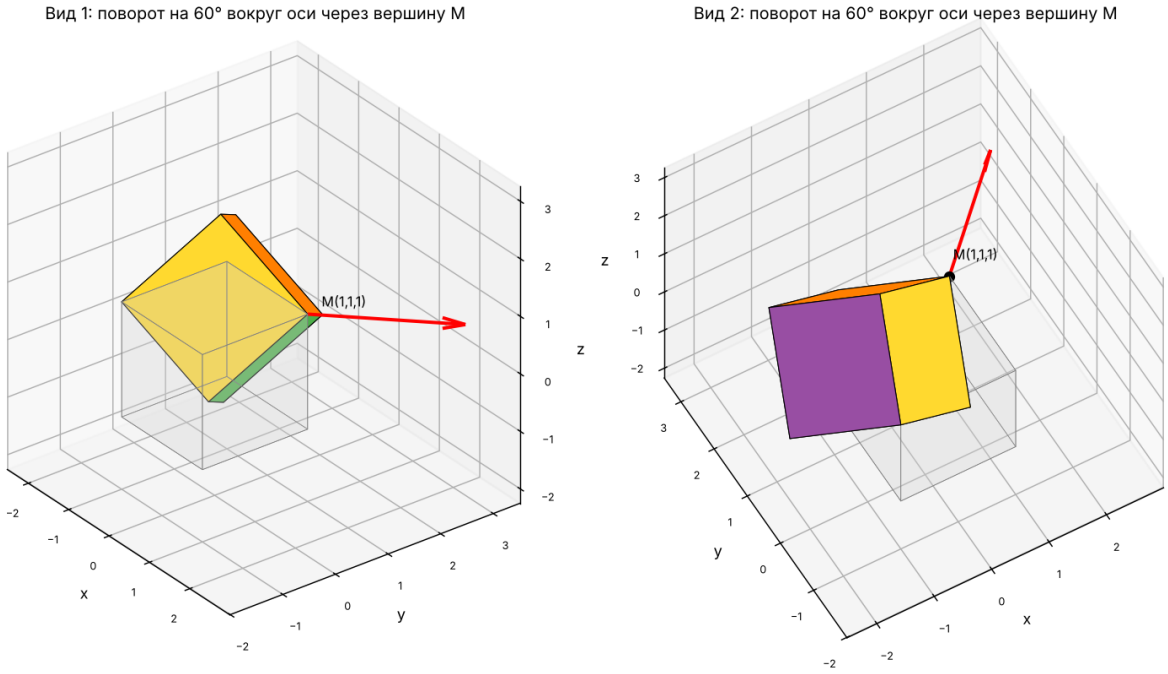


Рис. 8: Задание 5. Исходный и повёрнутый вокруг вершины $M(1, 1, 1)$ куб (два ракурса)

7 Задание 6. Камера

Сцена. Три куба, как на рисунке 1 методички: большой голубой $T(-3, 0, 0)\ S(2, 2, 2)$; синий (единичный) в начале координат; зелёный $T(3, 0, 0)\ R_z(45°)$. Все вершины умножаются на соответствующие матрицы. Сцена под стандартным углом камеры и при взгляде «снизу» показана на рисунках 9 и 11. Для вида «снизу» использованы `ax.view_init(azim=0, elev=-90)` и дополнительно `roll=-90`: в `matplotlib` без него ось x направлена вверх, а с ним картинка совпадает с рисунком 1 методички (x вправо, y вниз).

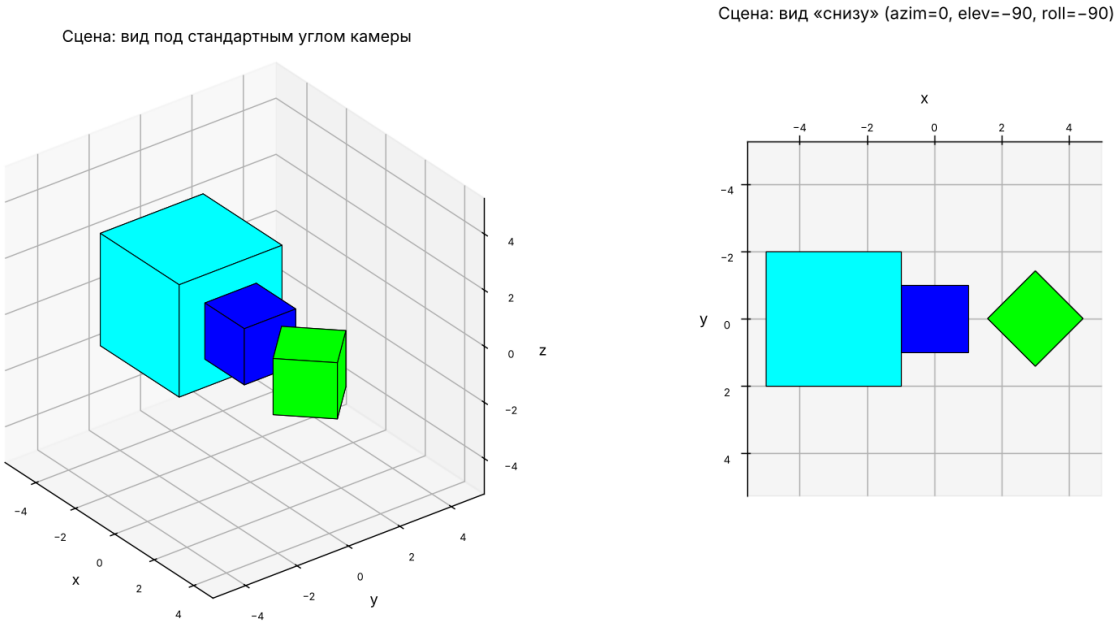


Рис. 9: Задание 6. Сцена под стандартным углом камеры (слева) и вид «снизу» (справа)

Положение камеры. Камера описывается матрицей $C = T_c R_c$, переводящей координаты камеры в мировые: R_c задаёт её поворот, T_c — положение. Выбрано $R_c = R_y(30°)R_x(25°)$ и расстояние $d = 12$ до центра сцены, камера смотрит в начало координат (направление взгляда $R_c e_z$), то есть её положение $c = -d\, R_c e_z = \begin{pmatrix} -5.438 \\ 5.071 \\ -9.419 \end{pmatrix}$:

$$R_c = \begin{pmatrix} 0.866 & 0.2113 & 0.4532 & 0 \\ 0 & 0.9063 & -0.4226 & 0 \\ -0.5 & 0.366 & 0.7849 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad T_c = \begin{pmatrix} 1 & 0 & 0 & -5.438 \\ 0 & 1 & 0 & 5.071 \\ 0 & 0 & 1 & -9.419 \\ 0 & 0 & 0 & 1 \end{pmatrix},$$

$$C = T_c R_c = \begin{pmatrix} 0.866 & 0.2113 & 0.4532 & -5.4378 \\ 0 & 0.9063 & -0.4226 & 5.0714 \\ -0.5 & 0.366 & 0.7849 & -9.4186 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Обратная матрица. $C^{-1} = R_c^{-1} T_c^{-1} = R_c^T T(-c)$, в блочном виде $\begin{pmatrix} R_c^T & -R_c^T c \\ 0 & 1 \end{pmatrix}$:

$$C^{-1} = \begin{pmatrix} 0.866 & 0 & -0.5 & 0 \\ 0.2113 & 0.9063 & 0.366 & 0 \\ 0.4532 & -0.4226 & 0.7849 & 12 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Проверка: $C^{-1}(c, 1)^T = (0, 0, 0, 1)^T$ (камера уходит в начало координат), а центр сцены $(0, 0, 0, 1)^T$ переходит в $(0, 0, 12, 1)^T$ — на ось z камеры на расстояние d. Применение C^{-1} ко всем вершинам даёт сцену в системе координат камеры (рисунок 10): эта матрица — жёсткое движение (поворот и перенос, $\det = 1$), форма и размеры объектов не меняются.

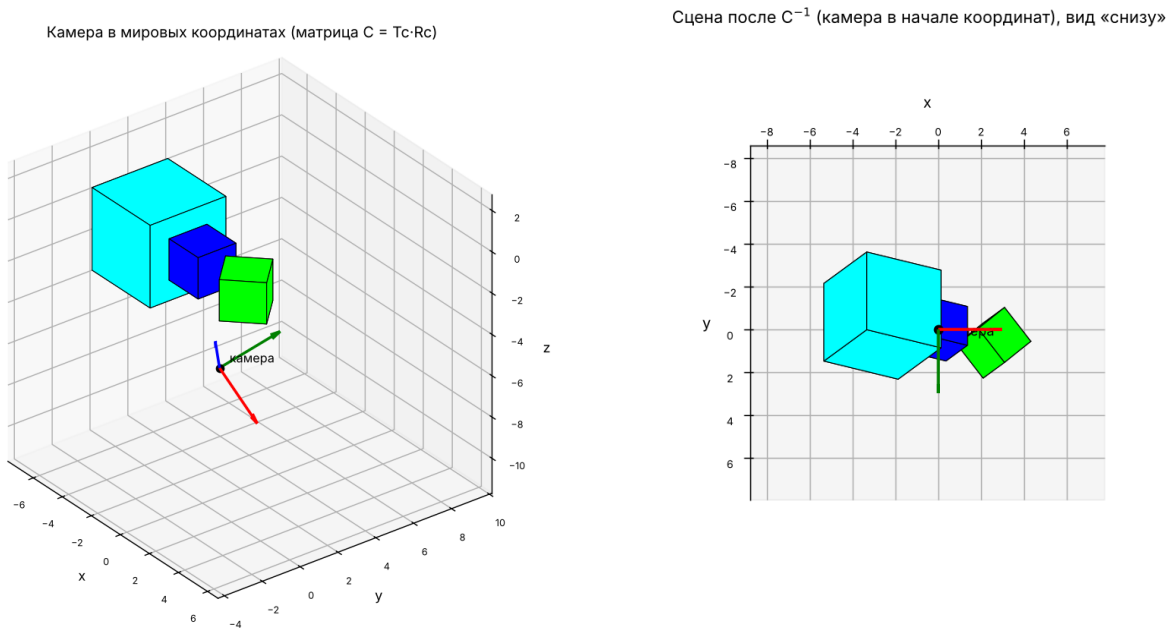


Рис. 10: Задание 6. Слева: камера (оси: x красная, y зелёная, z — направление взгляда, синяя) в мировых координатах. Справа: сцена после C^{-1}, вид «снизу» (с точки зрения камеры, расположенной в начале координат)

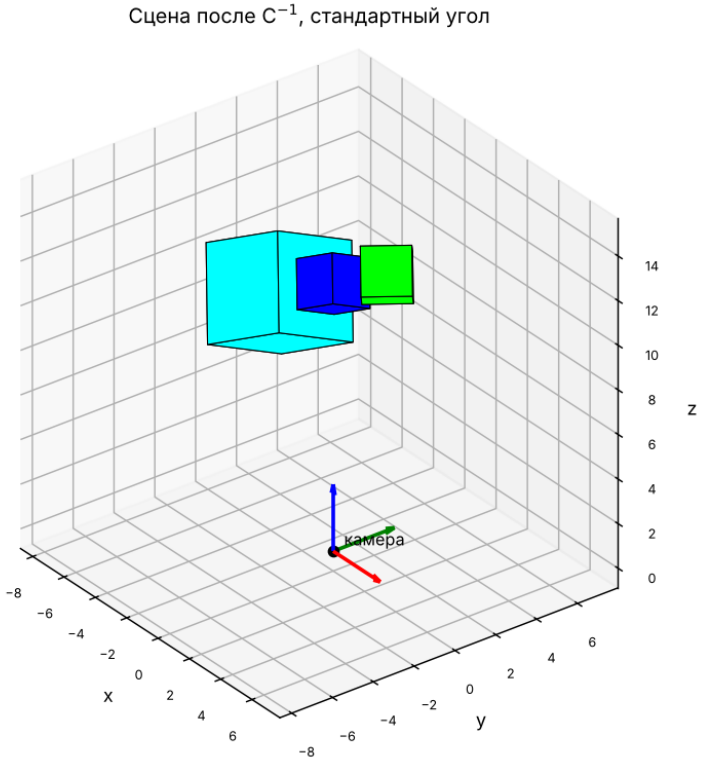


Рис. 11: Задание 6. Сцена после C^{-1} под стандартным углом: видно, что сцена сдвинута вдоль оси z камеры на расстояние $d = 12$

8 Задание 7. Перспектива

Устройство матрицы. Камера в начале координат смотрит вдоль $+z$. Видимая область — усечённая пирамида (frustum): ближняя плоскость $z = n$ с окном $[l, r] \times [b, t]$ и дальняя плоскость $z = f$. Матрица P переводит frustum в куб $[-1, 1]^3$. По подобию треугольников точка (x, y, z) проецируется на ближнюю плоскость в $x_p = nx/z$, $y_p = ny/z$, затем окно отображается на $[-1, 1]$: $x' = \dfrac{2x_p - (r + l)}{r - l}$ и аналогично y'. Глубина преобразуется как $z' = A + B/z$ с условиями $z'(n) = -1$, $z'(f) = 1$, откуда $A = \frac{f+n}{f-n}$, $B = -\frac{2fn}{f-n}$ (преобразование монотонно, порядок точек по глубине сохраняется). Все три формулы имеют общий знаменатель z, поэтому в однородных координатах

$$P = \begin{pmatrix} \dfrac{2n}{r-l} & 0 & -\dfrac{r+l}{r-l} & 0 \\ 0 & \dfrac{2n}{t-b} & -\dfrac{t+b}{t-b} & 0 \\ 0 & 0 & \dfrac{f+n}{f-n} & -\dfrac{2fn}{f-n} \\ 0 & 0 & 1 & 0 \end{pmatrix}, \qquad P \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} * \\ * \\ * \\ z \end{pmatrix}.$$

Четвёртая компонента равна z, и деление на неё (перспективное деление) даёт $x' = */z$ и т. д. Параметры для реализации: n, f (ближняя и дальняя плоскости) и границы окна l, r, b, t — либо, для симметричного окна, угол обзора FOV и отношение сторон aspect: $t = n \tan \frac{\text{FOV}}{2}$, $r = t \cdot$ aspect, $l = -r$, $b = -t$. (При $r = -l$, $t = -b$ третьи элементы первых двух строк равны нулю.) Точки с $z \le 0$ находятся за камерой и не имеют смысла для перспективы, поэтому все объекты должны находиться в диапазоне $n < z < f$.

Частный случай. FOV $= 60°$, aspect $= 1$, $n = 5$, $f = 25$: $t = r = 5 \tan 30° \approx 2.887$,

$$P = \begin{pmatrix} 1.7321 & 0 & 0 & 0 \\ 0 & 1.7321 & 0 & 0 \\ 0 & 0 & 1.5 & -12.5 \\ 0 & 0 & 1 & 0 \end{pmatrix},$$

$$\frac{2n}{r-l} = \frac{1}{\tan 30°} \approx 1.7321, \quad \frac{f+n}{f-n} = 1.5, \quad \frac{2fn}{f-n} = 12.5.$$

После C^{-1} вершины сцены имеют $z \in [7.32 \ldots 14.79] \subset (5, 25)$ и после PC^{-1} с делением на w лежат в кубе $[-1, 1]^3$ (максимальная по модулю координата равна 0.883). Проверка: ближняя плоскость $z = 5$ переходит в $z' = -1$, дальняя — в $z' = +1$, угол окна $(r, t, n) \mapsto (1, 1, -1)$.

Сравнение с изометрической камерой. Параллельная (изометрическая, `proj_type='ortho'`) проекция просто отбрасывает координату вдоль взгляда: размер объекта не зависит от расстояния до камеры, параллельные рёбра остаются параллельными. При перспективе кубы, расположенные дальше от камеры, уменьшаются (масштаб $\sim 1/z$): зелёный куб по сравнению с голубым стал заметно мельче, а рёбра, идущие вдоль взгляда, сходятся к точке схода.

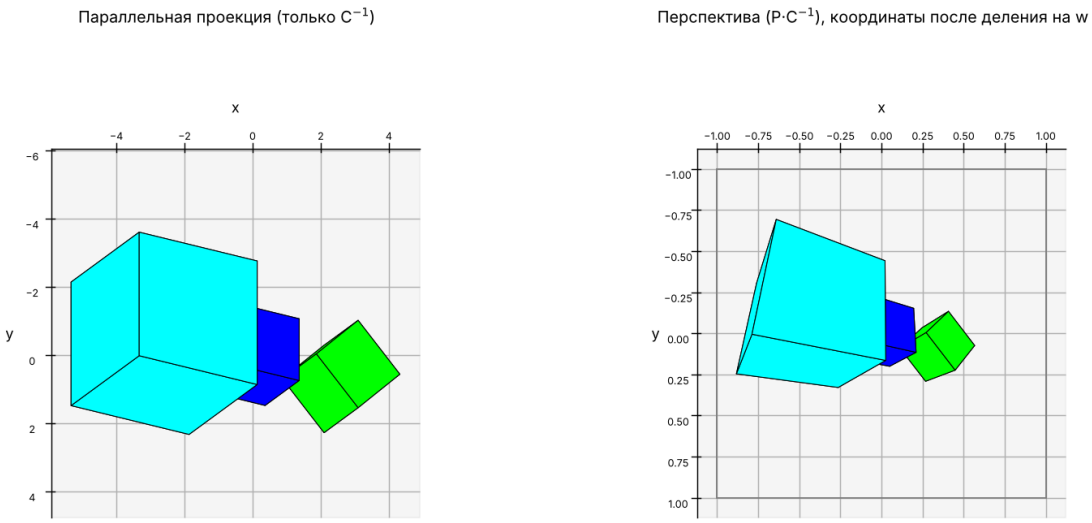


Рис. 12: Задание 7. Слева: параллельная проекция (только C^{-1}), справа: перспектива PC^{-1} после деления на w (серым — куб $[-1,1]^3$)

Деформация пространства. Рисунок 13 показывает ту же сцену до и после P в одном ракурсе: пирамида видимости превращается в куб, сходящиеся рёбра пирамиды становятся параллельными оси z, а объекты ближе к камере растягиваются (сильнее по z), дальше — сжимаются. Рисунок 14 — вращение стандартной камеры (`view_init`) для пространства после перспективы: видно, что расстояние по глубине распределено неравномерно (дальние объекты сильно сжаты по z'), а сами кубы деформируются в усечённые пирамиды.

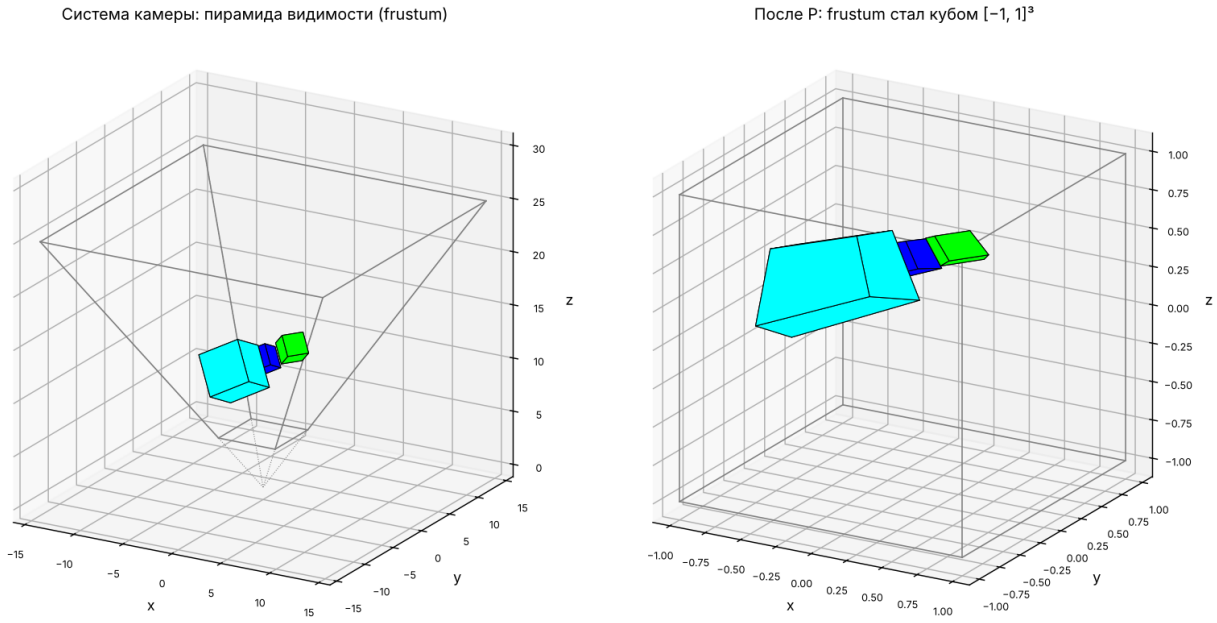


Рис. 13: Задание 7. Слева: сцена в системе камеры с пирамидой видимости ($n = 5$, $f = 25$, FOV $= 60°$). Справа: та же сцена после P — пирамида стала кубом $[-1,1]^3$

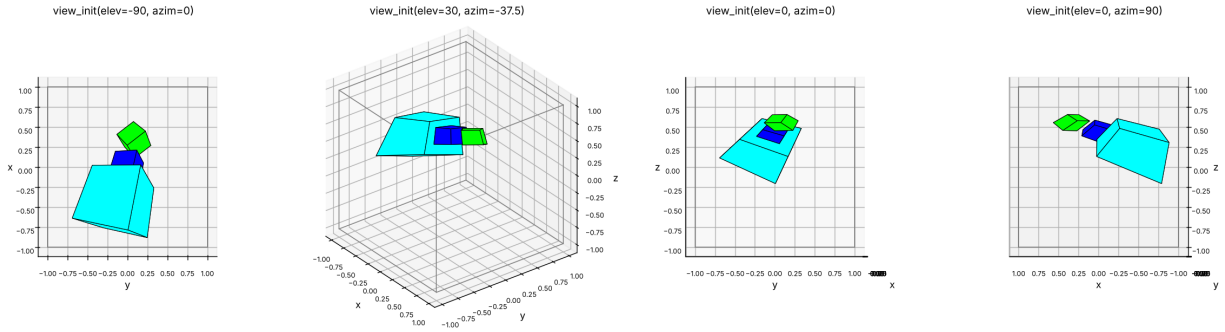


Рис. 14: Задание 7. Пространство после перспективы под разными углами: `view_init(elev=-90, azim=0)` (с `roll=-90`), `(30, -37.5)`, `(0, 0)`, `(0, 90)`

9 Задание 8 (необязательное). Домик

Домик построен из блоков, каждый получен матрицами масштабирования, поворота и перемещения из куба или призмы (в виде $T\,R\,S\,V$): стены — $T(0,0,1.5)\,S(2.5,2,1.5)$ от куба; крыша — $T(0,0,3)\,S(2.8,2.3,1.3)$ от треугольной призмы (вершины 4×6, грани: два треугольника и три четырёхугольника); труба — $T(1.5,0.8,3.8)\,S(0.3,0.3,0.8)$; дверь и два окна — тонкие параллелепипеды на передней стене; сарай рядом — $T(5.5,-0.5,0.75)\,R_z(30^\circ)\,S(1,1,0.75)$ с крышей из призмы; зелёная земля — плоский четырёхугольник. Камера направлена на домик сверху и спереди: $c=(-9,-13,9)$, взгляд в точку $(0.5,0,1.5)$, ось y камеры направлена вниз (против z мира), так что домик на виде «снизу» стоит правильно. Для неё

$$C = T_c R_c = \begin{pmatrix} 0.807 & -0.249 & 0.535 & -9 \\ -0.59 & -0.341 & 0.732 & -13 \\ 0 & -0.906 & -0.422 & 9 \\ 0 & 0 & 0 & 1 \end{pmatrix},$$

C^{-1} применяется ко всем объектам, затем перспектива $\mathrm{FOV}=60^\circ$, $n=5$, $f=40$:

$$P = \begin{pmatrix} 1.732 & 0 & 0 & 0 \\ 0 & 1.732 & 0 & 0 \\ 0 & 0 & 1.286 & -11.429 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Слева на рисунке — мировые координаты, посередине — вид параллельной камеры, справа — перспектива (углы домика сходятся, сарай, находящийся дальше, мельче).

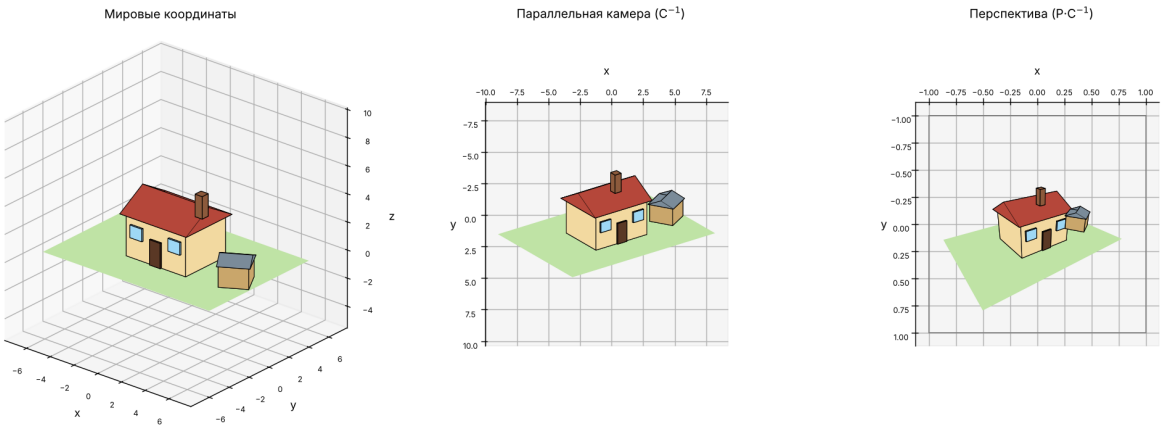


Рис. 15: Задание 8. Домик: мировые координаты, параллельная камера C^{-1} и перспектива PC^{-1}

10 Выводы

Все геометрические преобразования (масштабирование, перенос, поворот, смена камеры, перспектива) записаны матрицами 4×4 в однородных координатах, поэтому их композиция — обычное умножение матриц, причём порядок существенен: $TS \neq ST$, $R_2 R_1 \neq R_1 R_2$. Матрица поворота $R_v(\theta) = e^{J\theta}$ строится из кососимметрической матрицы векторного произведения, является ортогональной и оставляет неподвижной ось; поворот вокруг точки M — это сопряжение $T(M) R_v T(-M)$. Матрица камеры C^{-1} — жёсткое движение, переносящее камеру в начало координат; перспектива P нелинейна в $\mathbb{R}^3$, но линейна в однородных координатах, и после деления на $w = z$ превращает пирамиду видимости в куб. Правильность матриц проверена в программе численно (ортогональность, совпадение exp с формулой Родрига, неподвижность осей и центра вращения, образы ближней и дальней плоскостей).

11 Контрольные вопросы

11.1 Однородные координаты

Однородные координаты точки (x, y, z) — четвёрка (x, y, z, w), $w \neq 0$, определённая с точностью до общего множителя: $(x, y, z, w) \sim (\lambda x, \lambda y, \lambda z, \lambda w)$. Нужны, потому что в них перемещение (аффинное, а не линейное преобразование) и перспектива записываются матрицами, и любую цепочку преобразований можно заранее перемножить. Чтобы получить декартовы координаты, надо разделить на w: $(x/w,\ y/w,\ z/w)$. Векторы с $w = 0$ — направления (бесконечно удалённые точки).

11.2 Виды преобразований, аффинные преобразования

Основные виды: масштабирование, поворот, отражение, сдвиг (перемещение), скос (shear), проекции (параллельная и перспективная). Линейное преобразование: $p \mapsto Ap$ (оставляет начало координат на месте). Аффинное: $p \mapsto Ap + b$, то есть линейное плюс перемещение; оно сохраняет прямые, параллельность и отношение длин на прямой, но не фиксирует начало координат. В однородных координатах аффинному преобразованию соответствует матрица $\begin{pmatrix} A & b \\ 0 & 1 \end{pmatrix}$ с последней строкой $(0, 0, 0, 1)$. Перспектива — проективное, но не аффинное преобразование (последняя строка другая, параллельные прямые не сохраняются).

11.3 Масштабирование вдоль вектора

Для единичного u разложим $p = (u^T p)u + p_\perp$ и умножим только составляющую вдоль u на k: $S_u = I + (k-1)uu^T$ (при произвольном u: $uu^T/(u^Tu)$). Эквивалентно $S_u = Q\,\mathrm{diag}(k,1,1)\,Q^T$, где первый столбец $Q \in SO(3)$ равен u. В однородных координатах — блочная матрица $\mathrm{diag}(S_u, 1)$; если центр масштабирования — точка M, то $T(M)\,S_u\,T(-M)$.

11.4 Коммутирование S и T

S и T в общем случае не коммутируют: $ST = \begin{pmatrix} S & St \\ 0 & 1 \end{pmatrix}$, $TS = \begin{pmatrix} S & t \\ 0 & 1 \end{pmatrix}$, равенство только при $(S-I)t = 0$ (например, $t = 0$ или $S = I$). Геометрически: масштабирование выполняется относительно начала координат, поэтому если куб сначала сдвинуть, то и сдвиг будет масштабироваться. Две матрицы перемещения коммутируют: $T(a)T(b) = T(a+b) = T(b)T(a)$ (сдвиги складываются как векторы). Две матрицы масштабирования по осям коммутируют: диагональные матрицы перемножаются покоординатно. (Масштабирования вдоль неортогональных направлений, в общем случае, не коммутируют.)

11.5 Кососимметрические и симметрические матрицы

Кососимметрическая: $A^T = -A$. Её собственные значения нулевые или чисто мнимые, $\pm i\beta$ (если $Av = \lambda v$, то $v^*Av = \lambda\|v\|^2$ и одновременно $\overline{v^*Av} = v^*A^Tv = -v^*Av$, значит λ чисто мнимое). У J размера 3×3 с $\|v\| = 1$ собственные значения 0, $\pm i$; нулевое соответствует вектору v ($Jv = v \times v = 0$). У симметрических матриц ($A^T = A$) собственные значения вещественны, а собственные векторы различных значений ортогональны (спектральная теорема).

11.6 Комплексные собственные числа вещественной матрицы

Для вещественной матрицы комплексные собственные числа идут сопряжёнными парами $\lambda, \bar\lambda$, а соответствующие собственные векторы тоже комплексно сопряжены: $v, \bar v$. Вещественных собственных векторов для такой пары нет, но $\mathrm{Re}\,v$ и $\mathrm{Im}\,v$ порождают двумерное инвариантное подпространство, в котором матрица действует как поворот на угол $\arg\lambda$ с растяжением в $|\lambda|$ раз (блок $\begin{pmatrix} \alpha & -\beta \\ \beta & \alpha \end{pmatrix}$). Для вещественной матрицы нечётного порядка (например, 3×3) хотя бы одно собственное число вещественно.

11.7 Аналитические функции и матричная экспонента

Аналитическая функция — функция, которая локально разлагается в сходящийся степенной ряд $f(z) = \sum c_k z^k$ (в комплексном анализе — дифференцируемая в комплексном смысле). Матричная функция определяется подстановкой матрицы в ряд: $f(A) = \sum c_k A^k$; для диагонализуемой $A = V\Lambda V^{-1}$ получаем $f(A) = Vf(\Lambda)V^{-1}$ (функция применяется к собственным значениям), в общем случае используется жорданова форма. Матричная экспонента $e^A = \sum_{k=0}^{\infty} A^k/k!$ сходится для любой матрицы. Свойства: $e^{A+B} = e^Ae^B$ при $AB = BA$; $(e^A)^{-1} = e^{-A}$; $\det e^A = e^{\mathrm{tr}\,A}$; если $A^T = -A$, то e^A ортогональна. Применения: решение линейных систем $\dot x = Ax$, $x(t) = e^{At}x_0$; построение поворотов $R = e^{J\theta}$ из кососимметрических матриц (связь групп и алгебр Ли).

11.8 Собственное число 1 у матриц $SO(3)$

У любой $R \in SO(3)$ есть собственное значение 1: комплексные собственные значения образуют пару $e^{\pm i\theta}$, а $\det R = 1$ вынуждает третье равняться 1 (также у ортогональной матрицы нечётного порядка есть вещественное собственное число ± 1). Через экспоненту: $R = e^{J\theta}$, $Jv = 0$, поэтому $Rv = \sum \theta^k J^k v/k! = v$ — ось v неподвижна. Это содержание теоремы Эйлера о вращении: любое движение твёрдого тела вокруг неподвижной точки (любая матрица из $SO(3)$) — поворот вокруг некоторой оси на некоторый угол.

11.9 Ось вращения по матрице поворота

Ось — собственный вектор с $\lambda = 1$: решение $(R - I)v = 0$. Альтернатива: кососимметрическая часть $\frac{1}{2}(R - R^T) = \sin\theta\, J$, поэтому при $\sin\theta \neq 0$ ось пропорциональна $(R_{32} - R_{23},\ R_{13} - R_{31},\ R_{21} - R_{12})^T$ (так находятся оси композиций в задании 4), а угол из $\operatorname{tr} R = 1 + 2\cos\theta$. При $\theta = \pi$ матрица симметрична, $\frac{1}{2}(R + I) = \hat{v}\hat{v}^T$, и ось берётся из этой матрицы.

11.10 Подобные матрицы

A и B подобны, если $B = P^{-1}AP$ для некоторой невырожденной P: это одно и то же линейное отображение, записанное в разных базисах. Совпадают характеристический многочлен, собственные значения (с кратностями), след, определитель, ранг, жорданова форма. Различаются элементы матриц и собственные векторы (если $Av = \lambda v$, то $B(P^{-1}v) = \lambda P^{-1}v$). Пример из работы: $R_{v,M} = T(M)\, R_v\, T(M)^{-1}$ подобна R_v, поэтому оба поворота имеют одинаковые собственные значения $1, e^{\pm i\theta}, 1$, но ось и неподвижная точка разные.

11.11 Смысл матрицы C^{-1}

C^{-1} — переход от мировых координат к координатам камеры: вся сцена жёстко (поворотом и переносом, без деформации, $\det = 1$) двигается так, чтобы камера оказалась в начале координат с поворотом по умолчанию (смотрит вдоль $+z$). Это обратное к движению камеры преобразование: сдвинуть камеру вправо — то же, что сдвинуть сцену влево. Расстояния и углы между объектами не меняются, меняется лишь точка зрения.

11.12 Виды проекций и действие перспективы

В работе использованы параллельная (ортогональная, `proj_type='ortho'`) проекция — отбрасывание координаты вдоль взгляда, и перспективная (центральная). Перспектива геометрически переводит пирамиду видимости (frustum) в куб $[-1, 1]^3$: лучи, выходящие из центра камеры, становятся параллельными оси z, объекты уменьшаются пропорционально $1/z$, параллельные прямые, не параллельные плоскости изображения, сходятся в точке схода. Алгебраически после умножения на P четвёртая компонента становится равной z (а не 1), и деление на неё (перспективное деление) даёт нелинейную зависимость $x' \sim x/z$, $y' \sim y/z$, $z' = A + B/z$. При $z \to 0$ координаты уходят в бесконечность, точки за камерой ($z < 0$) имеют отрицательную w и отбрасываются.

12 Приложение. Код выполнения

Файл `prog/main.py` содержит построение всех матриц (масштабирование, перемещение, $R_v(\theta)$ через `scipy.linalg.expm` и формулу Родрига, поворот вокруг точки, камера, пер-

спектива), проверки (`assert`), генерацию рисунков в папку `graphics/` и числовых матриц для отчёта в `report/matrices.tex` (библиотеки `numpy`, `scipy`, `matplotlib`).

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""Лабораторная
 работа №3 — Матрицы в 3Dграфике-Практическая
 линейная алгебраСтуденты
: Ануфриев АС.. (465029), Бахуров НА.. (504862), Раджапов ФС.. (502544)
    Преподаватель
: Зарина ОА..Все

 преобразования координат выполняются матрицами 4x4 в однородных координатах.
    Соглашение
: векторстолбец-, p' = M p; композиция сначала" A, потом B" = B @ A.Камера
 смотрит вдоль оси +z, ось x направлена вправо, ось y внизвид
( снизу"": ax.view_init(azim=0, elev=-90)).Запуск

: python main.py из( любой папки). Графики сохраняются в ../graphics,числовые
 значения матриц для отчёта — в ../report/matrices.tex.
"""

from pathlib import Path

import numpy as np
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
from scipy.linalg import expm

ROOT = Path(__file__).resolve().parent.parent
GRAPHICS = ROOT / "graphics"
REPORT = ROOT / "report"
GRAPHICS.mkdir(parents=True, exist_ok=True)
REPORT.mkdir(parents=True, exist_ok=True)

# ============================================================================
# Задание 1. Фигуры: вершины (4 x N, однородные координаты) и грани
# ============================================================================
vertices_cube = np.array([
    [-1, 1, 1, -1, -1, 1, 1, -1],
    [-1, -1, 1, 1, -1, -1, 1, 1],
    [-1, -1, -1, -1, 1, 1, 1, 1],
    [1, 1, 1, 1, 1, 1, 1, 1],
], dtype=float)

faces_cube = [
    [0, 1, 5, 4],
    [1, 2, 6, 5],
    [2, 3, 7, 6],
    [3, 0, 4, 7],
    [0, 1, 2, 3],
    [4, 5, 6, 7],
]

# Другая фигура: четырёхугольная пирамида грани( разной длины: 1 квадрат + 4
   треугольника)
vertices_pyramid = np.array([
    [-1, 1, 1, -1, 0],
```

```
    [-1, -1, 1, 1, 0],
    [-1, -1, -1, -1, 1],
    [1, 1, 1, 1, 1],
], dtype=float)
faces_pyramid = [[0, 1, 2, 3], [0, 1, 4], [1, 2, 4], [2, 3, 4], [3, 0, 4]]

# Треугольная призма для( крыши домика): треугольник в плоскости y-z, вытянут вдоль x
vertices_prism = np.array([
    [-1, -1, -1, 1, 1, 1],
    [-1, 1, 0, -1, 1, 0],
    [0, 0, 1, 0, 0, 1],
    [1, 1, 1, 1, 1, 1],
], dtype=float)
faces_prism = [[0, 1, 2], [3, 4, 5], [0, 1, 4, 3], [1, 2, 5, 4], [2, 0, 3, 5]]

CUBE_FACE_COLORS = ["#e41a1c", "#4daf4a", "#ff7f00", "#984ea3", "#377eb8", "#ffd92f"]
PYRAMID_COLORS = ["#377eb8", "#e41a1c", "#4daf4a", "#ff7f00", "#984ea3"]

# Фигура = вершины(, грани, цветцвета/)
CUBE = (vertices_cube, faces_cube, CUBE_FACE_COLORS)


def draw_shape(ax, vertices, faces, color, alpha=1.0, edge="k", lw=0.6, zorder=None):
    Рисует""" многогранник. Деление на ю4- координату — переход к декартовым координатам."""
    pts = (vertices[:3, :] / vertices[3, :]).T
    polys = [pts[list(f)] for f in faces]
    coll = Poly3DCollection(polys, facecolors=color, edgecolors=edge, linewidths=lw, alpha=alpha)
    if zorder is not None:
        coll.set_zorder(zorder)
    ax.add_collection3d(coll)
    return pts


# ======================================================================
# Матрицы преобразований все( 4x4)
# ======================================================================
I4 = np.eye(4)


def S(sx, sy, sz):
    Масштабирование""" по осям."""
    return np.diag([sx, sy, sz, 1.0])


def T(tx, ty, tz):
    Перемещение""" сдвиг() на вектор (tx, ty, tz)."""
    M = np.eye(4)
    M[:3, 3] = [tx, ty, tz]
    return M


def J_matrix(v):
    Кососимметрическая""" матрица J = [v/|v|]_x, расширенная до 4x4."""
    x, y, z = np.asarray(v, float) / np.linalg.norm(v)
    return np.array([[0, -z, y, 0],
```

```
                      [z, 0, -x, 0],
                      [-y, x, 0, 0],
                      [0, 0, 0, 0]], dtype=float)


def Rv(v, theta):
    Поворот""" на угол theta вокруг оси v через начало координат: R = exp(J*
    theta)."""
    return expm(J_matrix(v) * theta)


def Rv_rodrigues(v, theta):
    То""" же по формуле Родрига: R = I + sin(t) J + (1 - cos(t)) J^2 на( блоке 3
    x3)."""
    J = J_matrix(v)
    R = np.eye(4) + np.sin(theta) * J + (1 - np.cos(theta)) * (J @ J)
    R[3, 3] = 1.0
    return R


def Rx(t):
    c, s = np.cos(t), np.sin(t)
    return np.array([[1, 0, 0, 0], [0, c, -s, 0], [0, s, c, 0], [0, 0, 0, 1]],
    float)


def Ry(t):
    c, s = np.cos(t), np.sin(t)
    return np.array([[c, 0, s, 0], [0, 1, 0, 0], [-s, 0, c, 0], [0, 0, 0, 1]],
    float)


def Rz(t):
    c, s = np.cos(t), np.sin(t)
    return np.array([[c, -s, 0, 0], [s, c, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1]],
    float)


def R_about_point(v, theta, M):
    Поворот""" вокруг оси с направлением v, проходящей через точку M: T(M) R_v T
    (-M)."""
    M = np.asarray(M, float)
    return T(*M) @ Rv(v, theta) @ T(*(-M))


def axis_angle(R):
    Ось""" и угол поворота по матрице задача( 9 из контрольных вопросов)."""
    R3 = R[:3, :3]
    theta = np.arccos(np.clip((np.trace(R3) - 1) / 2, -1, 1))
    w = np.array([R3[2, 1] - R3[1, 2], R3[0, 2] - R3[2, 0], R3[1, 0] - R3[0,
    1]])
    return w / np.linalg.norm(w), theta


def perspective(n, f, l, r, b, t):
    Матрица""" перспективы для камеры, смотрящей вдоль +z.
    Усечённая пирамида (frustum) [l,r]x[b,t] на плоскости z=n, дальняя
    плоскость z=f
    отображается в куб [-1,1]^3: z=n -> -1, z=f -> +1."""
    return np.array([
```

```
        [2 * n / (r - l), 0, -(r + l) / (r - l), 0],
        [0, 2 * n / (t - b), -(t + b) / (t - b), 0],
        [0, 0, (f + n) / (f - n), -2 * f * n / (f - n)],
        [0, 0, 1, 0],
    ], dtype=float)


def perspective_fov(fov_deg, aspect, n, f):
    t = n * np.tan(np.radians(fov_deg) / 2)
    r = t * aspect
    return perspective(n, f, -r, r, -t, t)


def look_at(c, target, up):
    Камера""" в точке c, смотрит на target; ось y камеры направлена вниз""
   против( up).
    Возвращает Tc, Rc: C = Tc @ Rc переводит координаты камеры в мировые."""
    c, target, up = (np.asarray(a, float) for a in (c, target, up))
    z = target - c
    z /= np.linalg.norm(z)
    y = -(up - (up @ z) * z)
    y /= np.linalg.norm(y)
    x = np.cross(y, z)
    Rc = np.eye(4)
    Rc[:3, :3] = np.c_[x, y, z]
    return T(*c), Rc


# ======================================================================
# Параметры заданий
# ======================================================================
deg = np.radians

# Задание 2
S1 = S(2, 1, 1.5)
S2 = S(0.5, 2, 2)
S12 = S2 @ S1                       # сначала S1, потом S2

# Задание 3
T1 = T(2, 1, 0)
T2 = T(-1, 0, 2)
T12 = T2 @ T1
TS = T12 @ S12                      # масштабирование, затем перемещение
ST = S12 @ T12                      # перемещение, затем масштабирование

# Задание 4
v1, th1 = np.array([1.0, 2.0, 2.0]), deg(60)
v2, th2 = np.array([2.0, -1.0, 2.0]), deg(45)
R1 = Rv(v1, th1)
R2 = Rv(v2, th2)
R21 = R2 @ R1                       # сначала R1, затем R2
R12 = R1 @ R2                       # сначала R2, затем R1

# Задание 5: вращение вокруг вершины куба
M_vertex = np.array([1.0, 1.0, 1.0])
v5, th5 = np.array([1.0, 1.0, 0.0]), deg(60)
R_vertex = R_about_point(v5, th5, M_vertex)

# Задание 6: сцена из трёх кубиков и камера
cube_cyan = (T(-3, 0, 0) @ S(2, 2, 2) @ vertices_cube, faces_cube, "cyan")
```

```
cube_blue = (vertices_cube, faces_cube, "blue")
cube_green = (T(3, 0, 0) @ Rz(deg(45)) @ vertices_cube, faces_cube, "lime")
scene = [cube_cyan, cube_blue, cube_green]

alpha_c, beta_c, dist_c = deg(25), deg(30), 12.0
Rc = Ry(beta_c) @ Rx(alpha_c)
cam_pos = -dist_c * Rc[:3, 2]       # камера смотрит в начало координат с расстояния dist_c
Tc = T(*cam_pos)
C = Tc @ Rc
Cinv = np.linalg.inv(C)

# Задание 7: перспектива
FOV, ASPECT, NEAR, FAR = 60.0, 1.0, 5.0, 25.0
P = perspective_fov(FOV, ASPECT, NEAR, FAR)

# ======================================================================
# Проверки
# ======================================================================
def check(cond, msg):
    assert cond, fОШИБКА": {msg}"
    print("OK  ", msg)


def to_cart(V):
    return V[:3] / V[3]


def run_checks():
    # Задание 2-3
    check(np.allclose(S12, np.diag([1, 2, 3, 1])), "S2 S1 = diag(1,2,3,1)")
    check(np.allclose(T12, T(1, 1, 2)), "T2 T1 = T(1,1,2): сдвиги складываются и коммутируют")
    check(np.allclose(T1 @ T2, T2 @ T1), "T1 T2 = T2 T1")
    check(np.allclose(S1 @ S2, S2 @ S1), "S1 S2 = S2 S1")
    t = T12[:3, 3]
    check(np.allclose(TS[:3, :3], S12[:3, :3]) and np.allclose(TS[:3, 3], t), "TS = [[S, t],[0,1]]")
    check(np.allclose(ST[:3, 3], S12[:3, :3] @ t), "ST = [[S, S t],[0,1]]")
    check(not np.allclose(TS, ST), "TS != ST")
    # Задание 4
    for v, th in [(v1, th1), (v2, th2), ([0, 0, 1], 1.0), ([1, 0, 0], 0.3)]:
        check(np.allclose(Rv(v, th), Rv_rodrigues(v, th)), f"exp(J t) совпадает с формулой Родрига, v={list(v)}")
    R3 = R1[:3, :3]
    check(np.allclose(R3.T @ R3, np.eye(3)) and np.isclose(np.linalg.det(R3), 1), "R1 ортогональна, det = 1")
    check(np.allclose(R1 @ np.r_[v1, 1], np.r_[v1, 1]), "R1 v1 = v1 ось( неподвижна)")
    check(np.allclose(Rv([1, 0, 0], 0.7), Rx(0.7)) and np.allclose(Rv([0, 1, 0], 0.7), Ry(0.7))
          and np.allclose(Rv([0, 0, 1], 0.7), Rz(0.7)), "Rv для осей x, y, z = Rx, Ry, Rz")
    check(not np.allclose(R21, R12), "R2 R1 != R1 R2")
    ax21, a21 = axis_angle(R21)
    check(np.allclose(R21 @ np.r_[ax21, 1], np.r_[ax21, 1]), ось" R2 R1 неподвижна ось( найдена по кососимметричной части)")
    ax12, a12 = axis_angle(R12)
    check(np.allclose(R12 @ np.r_[ax12, 1], np.r_[ax12, 1]), ось" R1 R2 неподвижна")
```

```
    check(np.isclose(np.trace(R21[:3, :3]), np.trace(R12[:3, :3])), "tr(R2R1) = tr(R1R2): одинаковый угол поворота")
    # Задание 5
    check(np.allclose(R_vertex @ np.r_[M_vertex, 1], np.r_[M_vertex, 1]), вершина" M неподвижна")
    check(np.allclose(R_vertex[:3, :3], Rv(v5, th5)[:3, :3]), блок" 3x3 равен R_v")
    check(np.allclose(R_vertex[:3, 3], M_vertex - Rv(v5, th5)[:3, :3] @ M_vertex), сдвиг": M - R M")
    check(np.allclose(np.sort(np.abs(np.linalg.eigvals(R_vertex))), 1), " R_vertex подобна R_v: все λ|| = 1")
    # Задание 6
    check(np.allclose(C @ Cinv, I4), "C C^-1 = I")
    check(np.allclose(Cinv, np.linalg.inv(Rc) @ np.linalg.inv(Tc)) and np.allclose(Rc[:3, :3].T, np.linalg.inv(Rc[:3, :3])),
          "C^-1 = Rc^T Tc^-1")
    check(np.allclose(Cinv @ np.r_[cam_pos, 1], [0, 0, 0, 1]), камера" переходит в начало координат")
    check(np.allclose(Cinv @ np.r_[0, 0, 0, 1], [0, 0, dist_c, 1]), центр" сцены уходит на ось z на расстояние d")
    # Задание 7
    allv = np.hstack([Cinv @ V for V, _, _ in scene])
    z = allv[2]
    check(z.min() > NEAR and z.max() < FAR, fвсе" вершины между плоскостями: z in [{z.min():.2f}, {z.max():.2f}], внутри ({NEAR}, {FAR})")
    ndc = to_cart(P @ allv)
    check(np.abs(ndc).max() <= 1.0, fпосле" перспективы все вершины в кубе [-1,1]^3 (max |coord| = {np.abs(ndc).max():.3f})")
    check(np.allclose(to_cart(P @ np.r_[0, 0, NEAR, 1]), [0, 0, -1]) and np.allclose(to_cart(P @ np.r_[0, 0, FAR, 1]), [0, 0, 1]),
          ближняя" плоскость -> z=-1, дальняя -> z=+1")
    tt = NEAR * np.tan(np.radians(FOV) / 2)
    check(np.allclose(to_cart(P @ np.r_[tt, tt, NEAR, 1]), [1, 1, -1]), угол" ближней плоскости frustum -> (1,1,-1)")
    check(np.isclose((P @ np.r_[1, 2, 7, 1])[3], 7), я"4- координата после P равна z перспективное( деление)")


# ======================================================================
# Рисование
# ======================================================================
VIEW_STD = dict(elev=30, azim=-37.5)     # стандартный вид из листинга
VIEW_BELOW = dict(elev=-90, azim=0, roll=-90)  # вид снизу"" как на рис. 1 методички (x вправо, y вниз)
GHOST = dict(color="lightgray", alpha=0.18, edge="gray", lw=0.4)


def new_fig(n, size=5.2, rows=1):
    cols = int(np.ceil(n / rows))
    fig = plt.figure(figsize=(size * cols, size * rows))
    axes = [fig.add_subplot(rows, cols, i + 1, projection="3d", proj_type="ortho") for i in range(n)]
    return fig, axes


def finish(ax, pts, title="", view=VIEW_STD, margin=0.12, ticks=True, lim=None):
    Равные""" масштабы по осям, пределы по облаку точек pts (N x 3)."""
    pts = np.vstack(pts)
    if lim is None:
        lo, hi = pts.min(0), pts.max(0)
```

```
        ctr, half = (lo + hi) / 2, (hi - lo).max() / 2 * (1 + margin)
        lim = [(c - half, c + half) for c in ctr]
    ax.set_xlim(*lim[0]); ax.set_ylim(*lim[1]); ax.set_zlim(*lim[2])
    ax.set_box_aspect([1, 1, 1])
    ax.view_init(**view)
    ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("z")
    ax.set_title(title, fontsize=11)
    if ticks:
        ax.tick_params(labelsize=7)
    if abs(view.get("elev", 0)) == 90:      # смотрим вдоль оси z: подписи z не нужны
        ax.set_zticks([])
        ax.set_zlabel("")


def draw_axis(ax, v, M=(0, 0, 0), L=2.6, color="red"):
    Вектор""", коллинеарный оси вращения со( стрелкой) и пунктир вдоль всей оси."""
    v = np.asarray(v, float) / np.linalg.norm(v)
    M = np.asarray(M, float)
    a, b = M - L * v, M + L * v
    ax.plot(*zip(a, b), "--", color=color, lw=1)
    ax.quiver(*M, *v, length=L, color=color, lw=2.2, arrow_length_ratio=0.15)
    return np.array([a, b])


def save(fig, name, dpi=130):
    fig.tight_layout()
    fig.savefig(GRAPHICS / f"{name}.png", dpi=dpi, bbox_inches="tight")
    plt.close(fig)
    print(f"Saved: {GRAPHICS / (name + '.png')}")


def pair(V, M, faces=faces_cube, colors=CUBE_FACE_COLORS, ghost=True):
    Рисует""" образ ·MV, опционально ghost — исходную фигуру; возвращает точки."""
    pts = []
    return pts


def panel(ax, objs, title, view=VIEW_STD, ghost_objs=(), extra_pts=(), **kw):
    """objs: список (V,faces,color) для рисования; ghost_objs — полупрозрачные исходные фигуры."""
    pts = []
    for V, faces, col in ghost_objs:
        pts.append(draw_shape(ax, V, faces, **{**GHOST, "color": GHOST["color"]}))
    for V, faces, col in objs:
        pts.append(draw_shape(ax, V, faces, col))
    pts += list(extra_pts)
    finish(ax, pts, title, view, **kw)
    return pts


def task1():
    # Результат кода из приложения: синий куб
    fig, (ax,) = new_fig(1, 5.5)
    draw_shape(ax, vertices_cube, faces_cube, "blue", edge="k", lw=0.8)
    ax.set_xlim(-1, 1); ax.set_ylim(-1, 1); ax.set_zlim(-1, 1)
    ax.set_box_aspect([1, 1, 1]); ax.view_init(azim=-37.5, elev=30)
```

```
    for setter in (ax.set_xticks, ax.set_yticks, ax.set_zticks):
        setter(np.linspace(-1, 1, 5))
    ax.set_titleКуб(" (vertices_cube, faces_cube)")
    save(fig, "t1_cube")
    # Другая фигура: пирамида
    fig, (ax,) = new_fig(1, 5.5)
    draw_shape(ax, vertices_pyramid, faces_pyramid, PYRAMID_COLORS)
    ax.set_xlim(-1, 1); ax.set_ylim(-1, 1); ax.set_zlim(-1, 1)
    ax.set_box_aspect([1, 1, 1]); ax.view_init(azim=-37.5, elev=30)
    for setter in (ax.set_xticks, ax.set_yticks, ax.set_zticks):
        setter(np.linspace(-1, 1, 5))
    ax.set_titleПирамида(" (vertices_pyramid, faces_pyramid)")
    save(fig, "t1_pyramid")


def task2():
    mats = [(I4, Исходный" куб"), (S1, "S1 = diag(2, 1, 1.5)"), (S2, "S2 = diag
    (0.5, 2, 2)"), (S12, "·S2S1 = diag(1, 2, 3)")]
    fig, axes = new_fig(4, 4.6, rows=1)
    allp = []
    for ax, (M, title) in zip(axes, mats):
        allp.append(M @ vertices_cube)
    lim_pts = [to_cart(V).T for V in allp]
    for ax, (M, title) in zip(axes, mats):
        panel(ax, [(M @ vertices_cube, faces_cube, CUBE_FACE_COLORS)], title,
              ghost_objs=[CUBE] if M is not I4 else (), extra_pts=lim_pts)
    save(fig, "t2_scale")


def task3():
    mats = [(T1, "T1 = T(2, 1, 0)"), (T2, "T2 = T−(1, 0, 2)"), (T12, "·T2T1 = T
    (1, 1, 2)")]
    fig, axes = new_fig(3, 4.6)
    lim = [to_cart(M @ vertices_cube).T for M, _ in mats]
    for ax, (M, title) in zip(axes, mats):
        panel(ax, [(M @ vertices_cube, faces_cube, CUBE_FACE_COLORS)], title,
    ghost_objs=[CUBE], extra_pts=lim)
    save(fig, "t3_translate")

    mats = [(TS, "TS: сначала масштаб, потом сдвиг"), (ST, "ST: сначала сдвиг,
    потом масштаб")]
    fig, axes = new_fig(2, 6)
    lim = [to_cart(M @ vertices_cube).T for M, _ in mats] + [to_cart(
    vertices_cube).T]
    for ax, (M, title) in zip(axes, mats):
        panel(ax, [(M @ vertices_cube, faces_cube, CUBE_FACE_COLORS)], title,
    ghost_objs=[CUBE], extra_pts=lim)
    save(fig, "t3_TS_ST")


def task4():
    # Две произвольные оси и углы, и их композиции двумя способами
    ax21, a21 = axis_angle(R21)
    ax12, a12 = axis_angle(R12)
    items = [
        (R1, v1, f"R1: v1=(1,2,2), θ°1=60"),
        (R2, v2, f"R2: v2−=(2,1,2), θ°2=45"),
        (R21, ax21, f"·R2R1 сначала( R1): θ≈{np.degrees(a21):.1f°}"),
        (R12, ax12, f"·R1R2 сначала( R2): θ≈{np.degrees(a12):.1f°}"),
    ]
```

```
    fig, axes = new_fig(4, 4.6)
    lim = [to_cart(M @ vertices_cube).T for M, _, _ in items] + [np.array([[-2.6] * 3, [2.6] * 3])]
    for ax, (M, v, title) in zip(axes, items):
        draw_axis(ax, v)
        panel(ax, [(M @ vertices_cube, faces_cube, CUBE_FACE_COLORS)], title, ghost_objs=[CUBE], extra_pts=lim)
    save(fig, "t4_rotations")

    # Зависимость от угла: фиксированная ось v1
    angles = [30, 90, 150, 180]
    fig, axes = new_fig(4, 4.6)
    for ax, a in zip(axes, angles):
        draw_axis(ax, v1)
        panel(ax, [(Rv(v1, deg(a)) @ vertices_cube, faces_cube, CUBE_FACE_COLORS)], f"v1=(1,2,2), θ = {a°}",
              ghost_objs=[CUBE], extra_pts=[np.array([[-2.6] * 3, [2.6] * 3])])
    save(fig, "t4_angles")

    # Зависимость от оси: Rx, Ry, Rz и диагональ (1,1,1) при θ = °45
    axes_list = [([1, 0, 0], "Rx°(45)"), ([0, 1, 0], "Ry°(45)"), ([0, 0, 1], "Rz°(45)"), ([1, 1, 1], "v=(1,1,1), θ°=45")]
    fig, axes = new_fig(4, 4.6)
    for ax, (v, title) in zip(axes, axes_list):
        draw_axis(ax, v)
        panel(ax, [(Rv(v, deg(45)) @ vertices_cube, faces_cube, CUBE_FACE_COLORS)], title,
              ghost_objs=[CUBE], extra_pts=[np.array([[-2.6] * 3, [2.6] * 3])])
    save(fig, "t4_axes")


def task5():
    fig, axes = new_fig(2, 6)
    for ax, view, title in zip(axes, [VIEW_STD, dict(elev=55, azim=-120)],
                               Вид[" 1", Вид" 2"]):
        line = draw_axis(ax, v5, M=M_vertex, L=2.4, color="red")
        ax.scatter(*M_vertex, color="black", s=40, zorder=20)
        ax.text(*(M_vertex + 0.15), "M(1,1,1)", fontsize=9)
        panel(ax, [(R_vertex @ vertices_cube, faces_cube, CUBE_FACE_COLORS)], f"{title}: поворот на °60 вокруг оси через вершину M",
              view=view, ghost_objs=[CUBE], extra_pts=[line, np.array([[-1.2] * 3, [1.2] * 3])])
    save(fig, "t5_vertex_rotation")


def camera_frame(ax, Cmat, size=2.5, label=True):
    Оси""" камеры: x — красная, y — зелёная, z направление( взгляда) — синяя."""
    o = Cmat[:3, 3]
    pts = [o]
    for k, col in enumerate(["red", "green", "blue"]):
        d = Cmat[:3, k]
        ax.quiver(*o, *d, length=size, color=col, lw=2, arrow_length_ratio=0.15)
        pts.append(o + size * d)
    ax.scatter(*o, color="black", s=30)
    if label:
        ax.text(*(o + 0.3), камера"", fontsize=9)
    return np.array(pts)


def scene_pts(objs, M=I4):
```

```
    return [to_cart(M @ V).T for V, _, _ in objs]


def transformed(objs, M):
    return [(M @ V, f, c) for V, f, c in objs]


def task6():
    # Два вида исходной сцены рисунки( 1-2 методички)
    fig, (ax,) = new_fig(1, 6)
    panel(ax, scene, Сцена": вид под стандартным углом камеры", view=VIEW_STD)
    save(fig, "t6_scene_standard")
    fig, (ax,) = new_fig(1, 6)
    panel(ax, scene, Сцена": вид «снизу» (azim=0, elev−=90, roll−=90)", view=
    VIEW_BELOW)
    save(fig, "t6_scene_below")

    # Камера в мировых координатах и сцена в системе камеры
    fig, (ax,) = new_fig(1, 6.5)
    cp = camera_frame(ax, C, size=3.0)
    panel(ax, scene, Камера" в мировых координатах матрица( C = ·TcRc)", view=
    VIEW_STD, extra_pts=[cp])
    save(fig, "t6_camera_world")

    fig, (ax,) = new_fig(1, 6)
    cp = camera_frame(ax, I4, size=3.0)
    panel(ax, transformed(scene, Cinv), Сцена" после C$^{-1}$ камера( в начале
    координат), вид «снизу»", view=VIEW_BELOW, extra_pts=[cp])
    save(fig, "t6_scene_camera")

    # Тот же результат под стандартным углом: видно, что сцена целиком сдвинута
    по оси z
    fig, (ax,) = new_fig(1, 6)
    cp = camera_frame(ax, I4, size=3.0)
    panel(ax, transformed(scene, Cinv), Сцена" после C$^{-1}$, стандартный
    угол", view=VIEW_STD, extra_pts=[cp])
    save(fig, "t6_scene_camera_std")


def frustum_lines(ax, n, f, fov, aspect=1.0, color="gray"):
    Рёбра""" усечённой пирамиды видимости в системе камеры."""
    t = np.tan(np.radians(fov) / 2)
    r = t * aspect
    pts = np.array([[sx * r * z, sy * t * z, z] for z in (n, f) for sx, sy in
    [(-1, -1), (1, -1), (1, 1), (-1, 1)]])
    edges = [(0, 1), (1, 2), (2, 3), (3, 0), (4, 5), (5, 6), (6, 7), (7, 4), (0,
     4), (1, 5), (2, 6), (3, 7)]
    for a, b in edges:
        ax.plot(*zip(pts[a], pts[b]), color=color, lw=0.9)
    o = np.zeros(3)
    for k in range(4, 8):
        ax.plot(*zip(o, pts[k]), color=color, lw=0.6, ls=":")
    return pts


def ndc_cube(ax, color="gray"):
    c = np.array([[sx, sy, sz] for sz in (-1, 1) for sx, sy in [(-1, -1), (1,
    -1), (1, 1), (-1, 1)]], float)
    edges = [(0, 1), (1, 2), (2, 3), (3, 0), (4, 5), (5, 6), (6, 7), (7, 4), (0,
     4), (1, 5), (2, 6), (3, 7)]
```

```
    for a, b in edges:
        ax.plot(*zip(c[a], c[b]), color=color, lw=0.9)
    return c


def task7():
    cam_scene = transformed(scene, Cinv)
    persp_scene = transformed(scene, P @ Cinv)

    # 1. Изометрическая параллельная() камера и перспектива, вид «снизу»
    fig, axes = new_fig(2, 6.2)
    panel(axes[0], cam_scene, Параллельная" проекция только( C$^{-1}$)", view=VIEW_BELOW)
    pc = ndc_cube(axes[1])
    panel(axes[1], persp_scene, Перспектива" (·PC$^{-1}$), координаты после деления на w", view=VIEW_BELOW, extra_pts=[pc])
    save(fig, "t7_ortho_vs_persp")

    # 2. Деформация пространства: frustum -> куб
    fig, axes = new_fig(2, 6.2)
    fr = frustum_lines(axes[0], NEAR, FAR, FOV)
    panel(axes[0], cam_scene, Система" камеры: пирамида видимости (frustum)", view=dict(elev=20, azim=-60), extra_pts=[fr])
    pc = ndc_cube(axes[1])
    panel(axes[1], persp_scene, После" P: frustum стал кубом −[1, ³1]", view=dict(elev=20, azim=-60), extra_pts=[pc])
    save(fig, "t7_frustum_to_cube")

    # 3. Вращение стандартной камеры для изучения деформации
    views = [dict(elev=-90, azim=0), dict(elev=30, azim=-37.5), dict(elev=0, azim=0), dict(elev=0, azim=90)]
    fig, axes = new_fig(4, 4.6)
    pc = ndc_cube(axes[0])
    for ax, view in zip(axes, views):
        ndc_cube(ax)
        panel(ax, persp_scene, f"view_init(elev={view['elev']}, azim={view['azim']})", view=view, extra_pts=[pc])
    save(fig, "t7_views")


# ------------------------------------------------------------- задание 8
def house():
    wall = (T(0, 0, 1.5) @ S(2.5, 2, 1.5) @ vertices_cube, faces_cube, "#f2d9a0")
    roof = (T(0, 0, 3) @ S(2.8, 2.3, 1.3) @ vertices_prism, faces_prism, "#b5473a")
    chimney = (T(1.5, 0.8, 3.8) @ S(0.3, 0.3, 0.8) @ vertices_cube, faces_cube, "#8c5a3c")
    door = (T(0, -2.05, 0.9) @ S(0.45, 0.06, 0.9) @ vertices_cube, faces_cube, "#5a3825")
    win1 = (T(-1.6, -2.05, 1.9) @ S(0.5, 0.06, 0.45) @ vertices_cube, faces_cube, "#9ed8f5")
    win2 = (T(1.6, -2.05, 1.9) @ S(0.5, 0.06, 0.45) @ vertices_cube, faces_cube, "#9ed8f5")
    shed = (T(5.5, -0.5, 0.75) @ Rz(deg(30)) @ S(1.0, 1.0, 0.75) @ vertices_cube, faces_cube, "#c9a66b")
    shed_roof = (T(5.5, -0.5, 1.5) @ Rz(deg(30)) @ S(1.1, 1.1, 0.5) @ vertices_prism, faces_prism, "#7a8b99")
    return [wall, roof, chimney, door, win1, win2, shed, shed_roof]

```

```

GROUND = np.array([[-7, 7, 7, -7], [-5, -5, 5, 5], [0, 0, 0, 0], [1, 1, 1, 1]],
    dtype=float)


def draw_house(ax, M, title, view, extra=()):
    ax.computed_zorder = False
    pts = [draw_shape(ax, M @ GROUND, [[0, 1, 2, 3]], "#bfe3a5", edge="none",
    zorder=1)]
    for V, faces, col in house():
        pts.append(draw_shape(ax, M @ V, faces, col, zorder=2))
    finish(ax, pts + list(extra), title, view)


def task8():
    c_h = np.array([-9.0, -13.0, 9.0])
    Tc_h, Rc_h = look_at(c_h, [0.5, 0, 1.5], [0, 0, 1])
    C_h = Tc_h @ Rc_h
    Ci_h = np.linalg.inv(C_h)
    P_h = perspective_fov(60, 1.0, 5, 40)
    # проверка: домик попадает в куб видимости
    allv = np.hstack([Ci_h @ V for V, _, _ in house()] + [Ci_h @ GROUND])
    ndc = to_cart(P_h @ allv)
    assert allv[2].min() > 5 and allv[2].max() < 40 and np.abs(ndc).max() <=
    1.0, домик" вне frustum"
    print("OK   домик внутри frustum (max |ndc| = %.3f)" % np.abs(ndc).max())

    fig, axes = new_fig(3, 5.4)
    draw_house(axes[0], I4, Мировые" координаты", dict(elev=25, azim=-50))
    draw_house(axes[1], Ci_h, Параллельная" камера (C$^{-1}$)", VIEW_BELOW)
    pc = ndc_cube(axes[2])
    draw_house(axes[2], P_h @ Ci_h, Перспектива" (·PC$^{-1}$)", VIEW_BELOW,
    extra=[pc])
    save(fig, "t8_house")
    return C_h, P_h


# ======================================================================
# Экспорт матриц для отчёта
# ======================================================================
def fnum(x, p=4):
    s = f"{x:.{p}f}".rstrip("0").rstrip(".")
    return "0" if s in ("-0", "") else s


def tex_mat(M, p=4):
    rows = r" \\ ".join(" & ".join(fnum(x, p) for x in row) for row in M)
    return r"\begin{pmatrix}" + rows + r"\end{pmatrix}"


def tex_vec(v, p=3):
    return r"\begin{pmatrix}" + r" \\ ".join(fnum(x, p) for x in v) + r"\end{
    pmatrix}"


def export_matrices(C_h, P_h):
    ax21, a21 = axis_angle(R21)
    ax12, a12 = axis_angle(R12)
    ndc = to_cart(P @ np.hstack([Cinv @ V for V, _, _ in scene]))
    z_cam = np.hstack([Cinv @ V for V, _, _ in scene])[2]
```

```
    defs = {
        "MatSa": tex_mat(S1), "MatSb": tex_mat(S2), "MatSab": tex_mat(S12),
        "MatTa": tex_mat(T1), "MatTb": tex_mat(T2), "MatTab": tex_mat(T12),
        "MatTS": tex_mat(TS), "MatST": tex_mat(ST),
        "MatJa": tex_mat(J_matrix(v1)[:3, :3], 4), "MatRa": tex_mat(R1), "MatRb": tex_mat(R2),
        "MatRba": tex_mat(R21), "MatRab": tex_mat(R12),
        "AxisBA": tex_vec(ax21), "AxisAB": tex_vec(ax12),
        "AngleBA": fnum(np.degrees(a21), 2), "AngleAB": fnum(np.degrees(a12), 2),
        "MatRvertex": tex_mat(R_vertex),
        "MatRc": tex_mat(Rc, 4), "MatTc": tex_mat(Tc, 3), "MatC": tex_mat(C, 4), "MatCinv": tex_mat(Cinv, 4),
        "CamPos": tex_vec(cam_pos, 3),
        "MatP": tex_mat(P, 4),
        "ZRange": f"{fnum(z_cam.min(), 2)}\\ldots {fnum(z_cam.max(), 2)}",
        "NDCmax": fnum(np.abs(ndc).max(), 3),
        "MatCh": tex_mat(C_h, 3), "MatPh": tex_mat(P_h, 3),
    }
    lines = ["% Файл создаётся автоматически программой prog/main.py — не править вручную"]
    for k, v in defs.items():
        lines.append(f"\\newcommand{{\\{k}}}{{{v}}}")
    (REPORT / "matrices.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")
    print("Saved:", REPORT / "matrices.tex")


def main():
    run_checks()
    print()
    task1(); task2(); task3(); task4(); task5(); task6(); task7()
    C_h, P_h = task8()
    export_matrices(C_h, P_h)
    np.set_printoptions(precision=4, suppress=True, linewidth=120)
    print("\nR1 =\n", R1, "\nR2 =\n", R2, "\nR_vertex =\n", R_vertex, "\nC =\n", C, "\nCinv =\n", Cinv, "\nP =\n", P)
    print("\nAll graphics saved to", GRAPHICS)


if __name__ == "__main__":
    main()
```

Листинг 1: Программа построения матриц и визуализации

13 Заключение

Выполнены задания 1–7 и необязательное задание 8, получены общие формулы и конкретные матрицы, их действие продемонстрировано на кубике, сцене из нескольких фигур и домике; даны ответы на все контрольные вопросы.